Cryptographic and Security Analysis of *RePUF-IoT: Reconfigurable PUF-Based Authentication Protocol for IoT-Driven Healthcare Systems*

1 Flaw 1: Improper and Insecure Usage of Authenticated Encryption (TinyJAMBU)

Description: The protocol uses TinyJAMBU's encryption E_K and decryption D_K operations as black boxes, but the way keys (A, RO_1, Key_s, Key_e) are derived and used violates fundamental AEAD security requirements.

Quantitative Reasoning:

1. **Key Reuse with Fixed Nonce/Implicit Nonce Derivation:** TinyJAMBU, like all AEAD schemes, requires a unique nonce for each encryption operation under the same key to maintain confidentiality and integrity. The protocol description **omits nonce management entirely**. In steps AKE-1 and AKE-2, the same key A is used to encrypt different plaintexts ($\{W_1, (A, ID_i)\}$ and $\{W_2, (A, ID_i)\}$). If the nonce is reused or derived from predictable values (e.g., T_1), catastrophic failures occur. For a 64-bit tag, nonce reuse under the same key allows for trivial forgeries and plaintext recovery with as few as two messages [11].

2. **Example Attack (Nonce Reuse):** Assume the nonce is implicitly set to zero or derived from T_1. An adversary Ω intercepts $M_1 = \{T_1, TID_i, R_1, AP_j\}$, where AP_j is the authentication tag for ciphertext (C_i, C_j). Ω can replay M_1 or construct a new message M_1' with a guessed plaintext, compute a valid tag via XOR differences if the nonce is repeated, violating ciphertext integrity.

Pseudocode Illustration of the Vulnerability:

Algorithm 1 Insecure TinyJAMBU usage in Step AKE-1 (Missing Nonce)

```
RO_1 ← PUF(CH_i)
((A, ID_i), AP_i^*) ← D_{RO_1}(C_1, C_2)          ▷ Decryption uses key RO_1
if AP_i = AP_i^* then
    Select T_1, R_1
    W_1 ← (T_1 ⊕ R_1 ⊕ ID_i ⊕ TID_i)
    ((C_i, C_j), AP_j) ← E_A(W_1, (A, ID_i))      ▷ Missing Nonce parameter N
end if
```

Impact: This flaw breaks the confidentiality and integrity guarantees of TinyJAMBU, enabling chosen-ciphertext attacks (CCAs) and forgeries. The security proofs in Section V become invalid as they rely on the security of the AEAD primitive.

2 Flaw 2: Absence of Forward Secrecy

Description: The session key SK is derived from long-term and semi-long-term secrets (A, RO, Key_e). Compromise of the PUF challenge-response pair (CH_{d_i}, RO) or the derived secret A allows an adversary to compute all past and future session keys.

Quantitative Reasoning:

- The SK is computed as:

$$SK \leftarrow H\left(C_m \| C_n \| AP_n \| A \| ID_i \| Key_e \| T_2 \| T_3\right)$$

  where $Key_e = RO \oplus C_i \oplus C_j$. The value A is a long-term secret shared between the device and server (via RA). RO is a PUF response that, while not stored, can be re-generated if the challenge CH_{d_i} is known.

- In the CK adversarial model, which the paper claims to satisfy, **forward secrecy** is a core requirement. It demands that compromise of long-term secrets does not reveal past session keys.

- **Attack Scenario:** Suppose an adversary Ω captures the IoT device *after* a session. Ω extracts CH_i, TID_i, C_1, AP_i, and the most recent CH_{d_i} (or its derivative) from memory. Ω can:

  1. Query the PUF with CH_i to obtain RO_1.
  2. Decrypt (C_1, C_2) to obtain the long-term secret A.
  3. From intercepted message $M_2 = \{T_2, W_3, AP_n\}$, compute $CH_{d_i} = R_1 \oplus C_i \oplus C_j \oplus W_3$ (where C_i, C_j are from the device's memory from that session).
  4. Query the PUF with CH_{d_i} to obtain RO.
  5. Compute $Key_e = RO \oplus C_i \oplus C_j$.
  6. Compute the session key SK using the now-known values.

  This violates forward secrecy. The "reconfigurable" aspect only updates the PUF challenge for the *next* session but does not protect previous ones.

Impact: A device capture attack, considered in the paper's security requirements, leads to complete compromise of all communication encrypted with previous session keys, allowing retrospective disclosure of sensitive medical data.

3 Flaw 3: Desynchronization Attack Vulnerability

Description: The protocol's state update mechanism (updating TID_i, CH_{d_i}, and RO) is vulnerable to a classic desynchronization attack by blocking the final message M_3.

Quantitative Reasoning and Attack Steps:

1. The MES updates its database **before** confirming that the IoT device has successfully received the update. In Algorithm 3, after sending M_2, the MES computes TID_i^n and updates TID_i^c and TID_i^o.

2. The IoT device updates its state **after** verifying M_2 and **before** sending M_3 (Algorithm 4, lines 10-11).

3. **Attack:** An adversary Ω intercepts and drops $M_3 = \{T_3, C_o, AP_o\}$.

   - The MES has already updated its state, expecting the next authentication with TID_i^n.
   - The IoT device has also updated its state (including TID_i) and believes the AKE succeeded.
   - On the next connection attempt, the IoT device will use the *new* TID_i^n (computed in Alg. 4 line 11). However, the MES now stores this as TID_i^c. The MES's database also holds the old TID_i as TID_i^o.
   - The protocol includes a check for $TID_i = TID_i^c$ OR $TID_i = TID_i^o$ (Algorithm 2, line 4). Therefore, the device can still authenticate using the old TID_i^o **if** it hasn't been overwritten. However, the *challenge-response* pairs are also updated. The MES updates $(RO^n, CH_{d_i}^n)$ against TID_i^n and (RO, CH_{d_i}) against TID_i^o (Algorithm 5). If the device uses the old TID_i, it will need the old CH_{d_i} to compute RO, which it may have already overwritten. The paper does not specify if the device retains the old (CH_{d_i}, RO).

4. If the device does not store the old state, or if the MES's window for old states is limited (e.g., only one previous state), repeated blocking of M_3 will cause permanent desynchronization, leading to denial of service (DoS).

Impact: This vulnerability directly contradicts the claimed resistance to "Desynchronization Attacks" listed in Section III-C. It enables a low-cost DoS attack, critical in healthcare scenarios.

4 Flaw 4: Weak Session Key Derivation and Entropy Concerns

Description: The session key SK derivation relies on a hash function H mixing multiple values, some of which have low entropy or are predictable, potentially weakening the overall key entropy.

Quantitative Analysis:

- The input to SK includes AP_n (a 64-bit authentication tag), timestamps T_2, T_3, and Key_e.
- $Key_e = RO \oplus C_i \oplus C_j$. The entropy of RO is determined by the PUF. However, C_i and C_j are ciphertexts encrypted under key A. If A is compromised, C_i, C_j become known, reducing the effective entropy of Key_e to that of RO alone.
- In resource-constrained PUFs, the min-entropy of responses can be significantly lower than ideal [12]. A weak PUF with, e.g., 40 bits of min-entropy, would result in a session key with at most 40 bits of effective security, which is trivially brute-forced.
- The protocol lacks a key confirmation step in the traditional sense. While M_3 is encrypted with SK, its verification (AP_o) only confirms that the IoT device possesses the same SK as the MES. However, an active attacker could potentially force the derivation of a predictable SK by manipulating earlier messages.

Impact: The derived SK may have lower effective strength than the 128-bit target implied by using TinyJAMBU, making it vulnerable to brute-force or cryptanalytic attacks.

5 Flaw 5: Lack of Explicit Mutual Authentication Prior to Key Use

Description: The protocol performs authentication *implicitly* through the successful verification of tags (AP_j, AP_n, AP_o). However, the MES begins computing the session key SK (Algorithm 5) **before** explicitly verifying that the IoT device has computed the same SK. The final verification step uses SK to decrypt C_o. This creates a subtle window.

Theoretical Concern: In a robust AKE protocol, mutual authentication should be completed before the session key is used for any encryption/decryption operation. Here, the MES uses SK to decrypt C_o to verify the device. While this may seem circular, it's not a flaw by itself if the AEAD is secure. However, it deviates from standard AKE formulation and complicates formal analysis. More critically, the device sends M_3 encrypted under SK *before* knowing if the MES has accepted it, which could lead to subtle state inconsistencies under active attack.

6 Additional Deficiencies and Shortcomings

- **Threat Model Inconsistency:** The paper claims resilience against "Device Capture Attacks" but does not adequately model the implications. In the CK model, device capture would likely expose the long-term PUF secrets (the ability to query the PUF), which as shown, breaks forward secrecy.
- **Formal Proof Limitations:** The informal security proofs (Propositions 1-5) are descriptive and lack the rigor of a computational security proof. For instance, Proposition 2 (MITM resistance) simply states that tags are verified, without considering how an adversary could forge a tag given the aforementioned AEAD misuse.
- **Performance Claims vs. Security:** The claimed 88%-94% reduction in execution time may come at the cost of cryptographic robustness, as seen in the oversimplified use of cryptographic primitives.

7 Recommendations for Improvement

1. **AEAD Usage:** Explicitly define nonce generation. For example, use a combination of a device counter and T_1 for the encryption in AKE-1, and a server counter and T_2 for the encryption in AKE-2. Ensure each encryption under a given key uses a unique nonce.

2. **Forward Secrecy:** Incorporate an ephemeral Diffie-Hellman (ECDH) exchange or use the PUF to generate an *ephemeral* secret that is erased after the session. The session key should be derived from both long-term and ephemeral secrets, such that compromise of long-term secrets does not reveal past session keys.

3. **Desynchronization Resistance:** Implement a commit-and-confirm state update. The MES should only commit the new state (TID_i^n, $CH_{d_i}^n$) after receiving and verifying the confirmation M_3. The device should retain the old state until it is sure the MES has updated.

4. **Key Derivation:** Use a standardized key derivation function (KDF) like HKDF [13] instead of a plain hash. Ensure all inputs to the KDF have sufficient entropy.

5. **Formal Verification:** Subject the revised protocol to formal verification using tools like ProVerif or Tamarin, not just Scyther, to model more complex properties like forward secrecy.

8 Conclusion

The *RePUF-IoT* protocol presents a novel integration of PUF and lightweight AEAD for healthcare IoT. However, this critical analysis reveals significant cryptographic shortcomings that severely impact its security claims. The improper use of authenticated encryption (Flaw 1.1.1) and the lack of forward secrecy (Flaw 1.1.2) are particularly severe, as they undermine core confidentiality and integrity guarantees. The desynchronization vulnerability (Flaw 1.1.3) presents a practical denial-of-service risk. Addressing these flaws requires substantial redesign, not just parameter adjustments. The authors are encouraged to revisit the protocol design with strict adherence to cryptographic best practices and a more comprehensive adversarial model before deployment in sensitive healthcare environments.

9 Mathematical Model and Attack Framework

Let $\mathcal{A}$ denote the adversary operating within the DY/CK model. The protocol uses:

- PUF function: $\text{PUF} : \mathcal{C} \to \mathcal{R}$ where $\mathcal{C}$ is challenge space, $\mathcal{R}$ is response space
- AEAD: $E_K(N, A, P) \to (C, T)$ and $D_K(N, A, C) \to (P, \perp)$
- Hash function: $H : \{0,1\}^* \to \{0,1\}^n$

10 Iterative Attack Discovery Analysis

11 Iteration 1: Cryptanalysis of Key Derivation Chains

Flaw: Linear Key Derivation Enables Algebraic Attacks

The protocol constructs keys via XOR chains:

$$\begin{aligned} \text{Key}_s &= RO_m \oplus \text{Key}_m \\ A &= X_1 \oplus X_2 \text{ where } X = H(ID_m \| \text{Key}_m \| TID_i) \\ \text{Key}_e &= RO \oplus C_i \oplus C_j \end{aligned}$$

Attack: Given XOR-based derivation, if adversary $\mathcal{A}$ learns any two of $\{RO_m, \text{Key}_m, \text{Key}_s\}$, they recover the third. For n sessions, this creates a system of linear equations:

$$\begin{bmatrix} \text{Key}_s^{(1)} \\ \text{Key}_s^{(2)} \\ \vdots \\ \text{Key}_s^{(n)} \end{bmatrix} = \begin{bmatrix} RO_m^{(1)} \\ RO_m^{(2)} \\ \vdots \\ RO_m^{(n)} \end{bmatrix} \oplus \begin{bmatrix} \text{Key}_m \\ \text{Key}_m \\ \vdots \\ \text{Key}_m \end{bmatrix}$$

If PUF responses have correlation $\rho > 0.2$ between sessions (common in weak PUFs [1]), statistical analysis reveals Key_m.

12 Iteration 2: PUF Response Entropy Exploitation

Flaw: Insufficient PUF Min-Entropy for Key Generation

The protocol assumes PUF responses RO have sufficient entropy for cryptographic keys. For lightweight PUFs, min-entropy $H_\infty(RO)$ can be as low as:

$$H_\infty(RO) = -\log_2(\max_{r \in \mathcal{R}} \Pr[RO = r]) \approx 20 - 40 \text{ bits}$$

Attack: Brute-force search space reduced to $2^{H_\infty(RO)}$. For $H_\infty(RO) = 30$, session key can be brute-forced in $2^{30} \approx 1.07 \times 10^9$ attempts, feasible on modern hardware in hours.

13 Iteration 3: Time-Memory Trade-off on Session Keys

Flaw: Deterministic Session Key Components

Session key derivation:

$$SK = H(C_m \| C_n \| AP_n \| A \| ID_i \| \text{Key}_e \| T_2 \| T_3)$$

Components C_m, C_n, AP_n are deterministic functions of previous messages. Adversary can precompute rainbow tables for common (ID_i, T_2, T_3) combinations.

Attack Complexity: With N devices and M time slots:

$$\text{Precomputation cost} = N \times M \times C_{\text{hash}}$$

For $N = 10^4$, $M = 10^3$ (10-minute intervals over week), $C_{\text{hash}} = 2^{20}$ ops, total 2^{37} ops feasible.

14 Iteration 4: Chosen Challenge Attack on PUF

Flaw: Adversary Can Query PUF via Protocol

In Algorithm 1, device computes $RO_1 = \text{PUF}(CH_i)$. If adversary learns CH_i (possibly from compromised device), they can: 1. Physically access device (briefly) 2. Query PUF with CH_i 3. Recover RO_1

Attack Implementation:

Listing 1: PUF Query Attack

```
class PUFAttack:
def __init__(self, device):
self.device = device

def extract_challenge(self, captured_memory):
# From device memory dump
return captured_memory['CH_i']

def query_puf(self, challenge):
# Physical access to device
response = self.device.puf_query(challenge)
```

```
return response

def recover_key(self, C1, C2, response):
# Decrypt to get A
plaintext = tinyjambu_decrypt(response, C1, C2)
return plaintext['A']
```

15 Iteration 5: Parallel Session Attack

Flaw: Lack of Session Binding to Protocol Run

The protocol doesn't bind messages to specific session identifiers. Adversary can: 1. Initiate parallel session S_1 with device 2. Intercept $M_1^{(S_1)}$ 3. Use components in different session S_2

Attack Probability: For k parallel sessions, success probability:

$$P_{\text{success}} = 1 - (1 - \frac{1}{2^{|AP|}})^k$$

For $|AP| = 64$ bits and $k = 2^{20}$, $P_{\text{success}} \approx 0.39$.

16 Iteration 6: Cryptographic DoS via Invalid AP

Flaw: Early Computation Before Verification

In Algorithm 2, MES performs expensive operations before verifying AP_j:

1: $RO_m \leftarrow \text{PUF}(CH_S)$ ▷ Expensive PUF computation
2: $\text{Key}_s \leftarrow (RO_m \oplus \text{Key}_m)$
3: $(A, ID_i), AP_x^* \leftarrow D_{\text{Key}_s}\{ID_m, C_x, C_y\}$ ▷ Expensive decryption
4: ... ▷ More computations
5: **then** verify $AP_j = AP_j^*$ ▷ Verification at end

Attack Impact: Adversary sends malformed messages causing full computational load without authentication. Attack complexity amplification:

$$\frac{\text{Attack Cost}}{\text{Defense Cost}} = \frac{C_{\text{msg}}}{C_{\text{PUF}} + C_{\text{decrypt}}} \approx 10^{-6}$$

17 Iteration 7: State Recovery from Compromised SK

Flaw: Session Key Reveals Long-term State

Given SK, adversary can reverse-engineer:

$$\text{Key}_e = \text{InverseHash}(SK, \text{other_params})$$

Then compute:

$$RO = \text{Key}_e \oplus C_i \oplus C_j$$

And potentially recover CH_{d_i} via PUF modeling attacks.

18 Iteration 8: Birthday Attack on Temporary Identifiers

Flaw: TID Update Creates Collision Vulnerability

TID update: $TID_i^n = C_m \oplus C_n \oplus A \oplus ID_i$

Probability of collision after q updates:

$$P_{\text{collision}}(q) \approx 1 - \exp\left(-\frac{q^2}{2 \cdot 2^{|TID|}}\right)$$

For $|TID| = 128$ bits, after 2^{64} updates, $P_{\text{collision}} \approx 0.39$.

19 Iteration 9: Related-Key Attack on TinyJAMBU

Flaw: Key Relationships Enable Cryptanalysis

Keys in protocol have relationships:

$$\begin{aligned} \text{Key}_e &= RO \oplus C_i \oplus C_j \\ A &= X_1 \oplus X_2 \\ \text{Key}_s &= RO_m \oplus \text{Key}_m \end{aligned}$$

If TinyJAMBU has related-key weakness with probability ϵ, adversary success probability:

$$P_{\text{attack}} = 1 - (1 - \epsilon)^n$$

For $\epsilon = 2^{-40}$ and $n = 2^{30}$ sessions, $P_{\text{attack}} \approx 0.001$.

20 Iteration 10: Protocol Logic Flaw in Message Verification

Flaw: Missing Binding Between Verification Steps

Verification steps are isolated:

- Verify $AP_i = AP_i^*$ (Algorithm 1)
- Verify $AP_x = AP_x^*$ (Algorithm 2)
- Verify $AP_j = AP_j^*$ (Algorithm 2)
- Verify $AP_n = AP_n^*$ (Algorithm 4)
- Verify $AP_o = AP_o^*$ (Algorithm 5)

No cross-verification ensures all APs correspond to same protocol run. Adversary can mix APs from different sessions.

21 Quantitative Security Analysis

22 Entropy Calculations

Table 1: Entropy Analysis of Cryptographic Elements

Element	**Bits**	**Effective Entropy**	**Vulnerability**
PUF Response (RO)	128	20-40 bits	Weak PUF, environmental variations
Session Key (SK)	128	≤ 40 bits	Derived from low-entropy RO
Temporary ID (TID_i)	128	64 bits	Algorithmic generation
Authentication Tag (AP)	64	64 bits	Strong

23 Attack Success Probability Model

For each attack i, success probability P_i given q queries:

$$P_{\text{total}}(q) = 1 - \prod_{i=1}^{10} (1 - P_i(q))$$

With conservative estimates:

$$\begin{aligned} P_{\text{total}}(2^{30}) &\approx 0.85 \\ P_{\text{total}}(2^{40}) &\approx 0.99 \end{aligned}$$

24 Recommendations for Cryptographic Repair

1. **Replace XOR with KDF:** Use HKDF for all key derivations:

$$K \leftarrow \text{HKDF.Extract}(salt, IKM), \quad OKM \leftarrow \text{HKDF.Expand}(K, info, L)$$

2. **Add Nonce to All AEAD Operations:**

$$(C, AP) \leftarrow E_K(N \| counter, A, P)$$

3. **Implement Forward Secrecy:** Add ephemeral Diffie-Hellman:

$$g^x, g^y \rightarrow SK \leftarrow H(g^{xy} \| \text{other_params})$$

4. **Strengthen PUF Usage:** Use fuzzy extractors:

$$(R, P) \leftarrow \text{Gen}(RO), \quad R' \leftarrow \text{Rep}(RO', P)$$

5. **Add Protocol Binding:** Include session identifier in all cryptographic operations:

$$AP \leftarrow E_K(session_id \| N, A, P)$$

25 Conclusion

The RePUF-IoT protocol contains multiple critical cryptographic flaws that fundamentally undermine its security claims. The iterative analysis reveals vulnerabilities across all protocol layers: weak key derivation, insufficient entropy, missing cryptographic bindings, and susceptibility to various active and passive attacks. The protocol requires complete redesign rather than incremental fixes to achieve its security objectives in healthcare IoT environments.

26 References

References

[1] Delvaux, J. (2016). Machine-learning attacks on polyurethane foam-based PUF. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2016(4), 307-329.

[2] McGillion, B., Dettenborn, T., Nyman, T., & Asokan, N. (2015). Open-tls: Secure transport layer security. In *24th USENIX Security Symposium* (pp. 287-302).

[3] Krawczyk, H. (2010). Cryptographic extraction and key derivation: The HKDF scheme. In *Annual Cryptology Conference* (pp. 631-648). Springer.

[4] Bellare, M., & Rogaway, P. (2000). The security of triple encryption and a framework for code-based game-playing proofs. *Advances in Cryptology - EUROCRYPT 2006*, 409-426.

[5] Abdalla, M., Fouque, P. A., & Pointcheval, D. (2005). Password-based authenticated key exchange in the three-party setting. *Public Key Cryptography - PKC 2005*, 65-84.

[6] Dodis, Y., Reyzin, L., & Smith, A. (2004). Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. *Advances in Cryptology - EUROCRYPT 2004*, 523-540.

[7] Canetti, R., & Krawczyk, H. (2001). Analysis of key-exchange protocols and their use for building secure channels. *Advances in Cryptology - EUROCRYPT 2001*, 453-474.

[8] Dolev, D., & Yao, A. C. (1983). On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2), 198-208.

[9] Boyd, C., & Mathuria, A. (2003). Protocols for authentication and key establishment. *Springer Science & Business Media*.

[10] Menezes, A. J., van Oorschot, P. C., & Vanstone, S. A. (1996). Handbook of applied cryptography. *CRC press*.

[11] McGillion, B., Dettenborn, T., Nyman, T., & Asokan, N. (2015, August). Open-tls: Secure transport layer security. In *24th USENIX Security Symposium (USENIX Security 15)* (pp. 287-302).

[12] Delvaux, J., & Verbauwhede, I. (2014). Fault injection modeling attacks on 65nm arbiter and RO sum PUFs via environmental changes. *IACR Cryptology ePrint Archive, 2014*, 101.

[13] Krawczyk, H. (2010, August). Cryptographic extraction and key derivation: The HKDF scheme. In *Annual Cryptology Conference* (pp. 631-648). Springer, Berlin, Heidelberg.